

Facultad de Ingeniería de la Universidad de Buenos Aires
Sistemas distribuidos 1 (75.74)

Documento de arquitectura

Nombre	Padrón
Rodrigo Martín Rotondo Adrada	109210
Santiago Tomás Testa	108301
Santiago Henseler	110732

Índice

Índice **2**
Alcance **2**
Objetivos y restricciones de la arquitectura **2**
Casos de uso **3**
Vista Lógica **4**
Vista de Procesos **6**
Vista de Desarrollo **9**
Vista de Despliegue **10**
Diagrama de Robustez **11**
Listado de tareas **12**
Anexo: Conteo y total de palabras **13**

Alcance

Este documento detalla la arquitectura planteada para analizar un conjunto de artículos obtenidos de Wikipedia en los idiomas Francés e Inglés, obtenidos del dataset Wikipedia Structured Contents, según los siguientes objetivos y restricciones.

Objetivos y restricciones de la arquitectura

Objetivos:

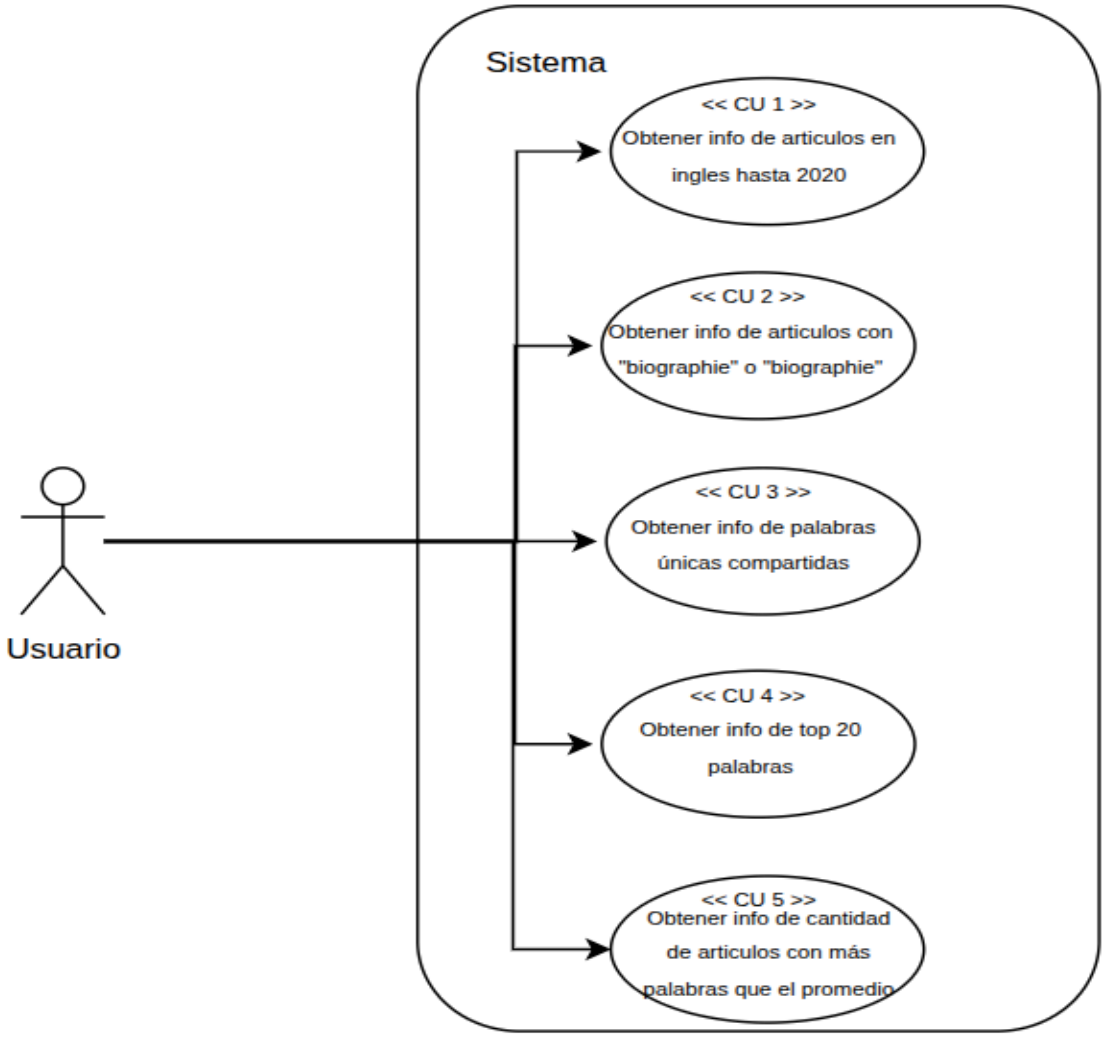
- Diseño escalable, distribuido y que aproveche los entornos multicomputadora.
- Soportar incremento de cómputo para escalar respecto a la cantidad de clientes atendidos y al volúmen de información a procesar por cliente.
- Aislamiento entre clientes.
- Evitar el procesamiento duplicado de datos, optimizando la utilización del sistema.

Restricciones:

- Arquitectura con un diseño específico para la resolución de las queries del enunciado (no resuelve queries genéricas, ni está abierto a dicha extensión).
- El diseño no considera elasticidad
- No fue considerada la posibilidad de extender a otros idiomas

Casos de uso

Se plantea como problema a resolver el siguiente caso de uso.



Siendo los siguientes casos de uso:

1. Url y fecha de modificación para artículos escritos en inglés hasta 2020 inclusive.
2. Nombre, url e imágen (bytes) de artículos escritos hasta 2020 inclusive que incluyan “biography” o “biographie” entre sus secciones.
3. Listado de palabras únicas compartidas entre cualquier artículo escrito en inglés y cualquier artículo escrito en francés.
4. Pares (palabra, cantidad) del top 20 palabras más utilizadas entre todos los abstracts. Dentro de un mismo abstract las palabras se cuentan una sola vez y las palabras triviales (stopwords) se excluyen del top
5. Url y cantidad de palabras de abstract para los artículos escritos a partir del 2021 cuya cantidad de palabras en el abstract sea superior al promedio de palabras de abstract de todos los artículos escritos hasta el 2020 inclusive. Se aplica el mismo criterio para contabilizar palabras que la query anterior.

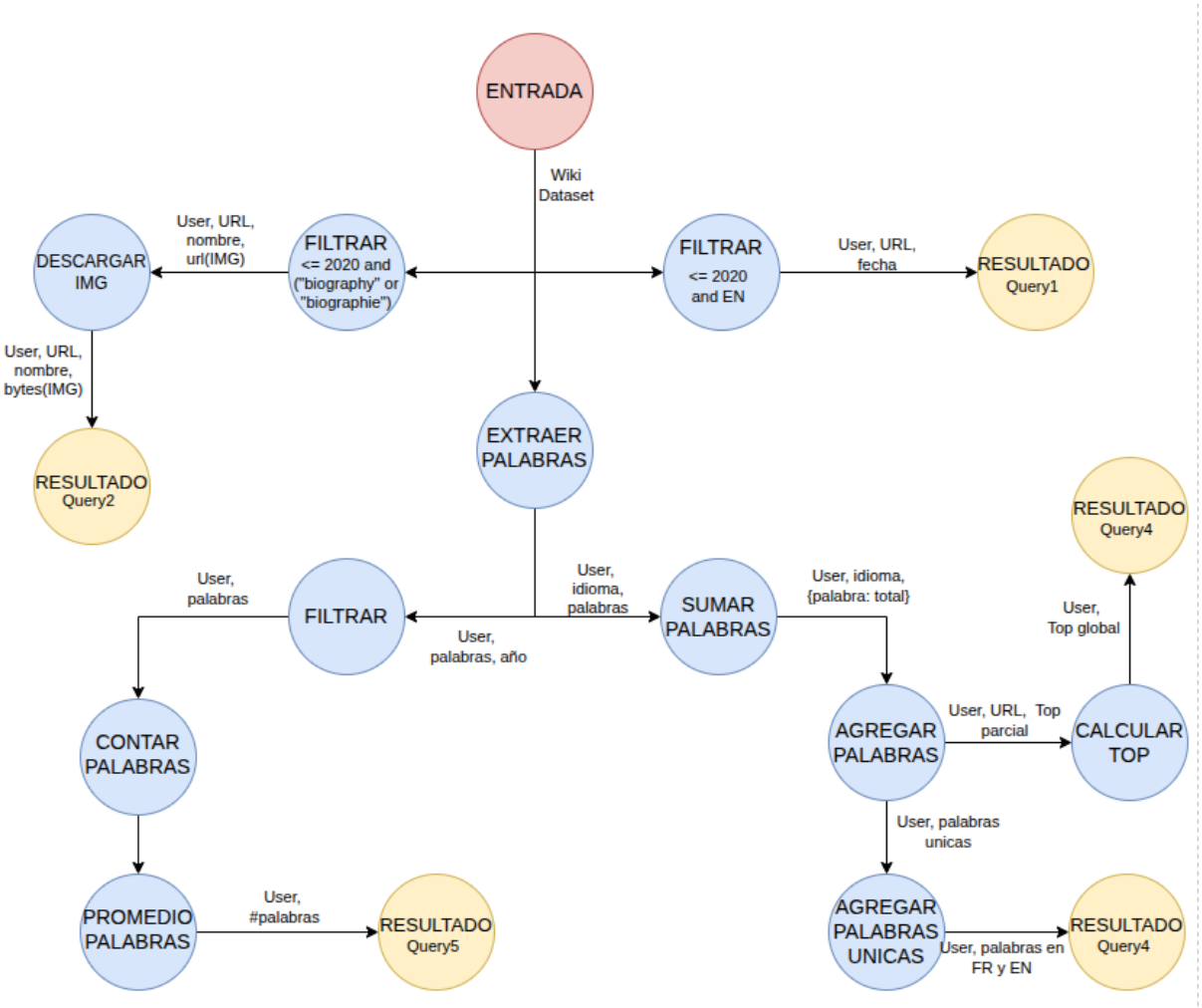
Cada uno de estos casos de usos van a ser identificados en el resto del documento como Query X, siendo X el subcaso de uso correspondiente (1 ... 5).

Vista Lógica

El siguiente diagrama busca explicar el flujo de datos dentro del sistema. Las primeras dos queries son respondidas en la primera etapa (clasificación). Luego pasa a la extracción de palabras según el criterio del enunciado, reduciendo la cantidad de datos a procesar en las siguientes etapas.

La primera ramificación obtiene el top de palabras con un esquema SUM-AGGREGATE-JOIN, resolviendo la query 4. De dicha etapa de aggregate se obtienen las palabras sin repetir de los abstract en inglés y francés del cliente, habilitando la búsqueda de coincidencias para la query 3.

Por otro lado, la segunda ramificación realiza la suma de palabras y almacena URL-cantidad de palabras (si la fecha es mayor a 2020) o la cantidad de abstracts y suma total de palabras (si la fecha es menor o igual 2020) tal de poder realizar el promedio .



Vista de Procesos

El sistema contará de los siguientes procesos:

1. Comunicación con cliente-gestión de EOFs
2. Preprocesamiento (transformador y filtro inicial)
3. Descargador de imágenes
4. Resolución de la tercera, cuarta y quinta query

En el siguiente diagrama se detalla el flujo de comunicación cliente-gateway, el preprocesamiento de abstracts en palabras y la descarga de imágenes. El código de colores es azul para retornos de loops, rosa para secuencias normales (dependencias directas con línea fija y dependencias indirectas con líneas punteadas), rojo para envío de respuestas al gateway y verde para cierre por sigterm (explícito únicamente para el gateway, aunque es implementado en los distintos componentes).

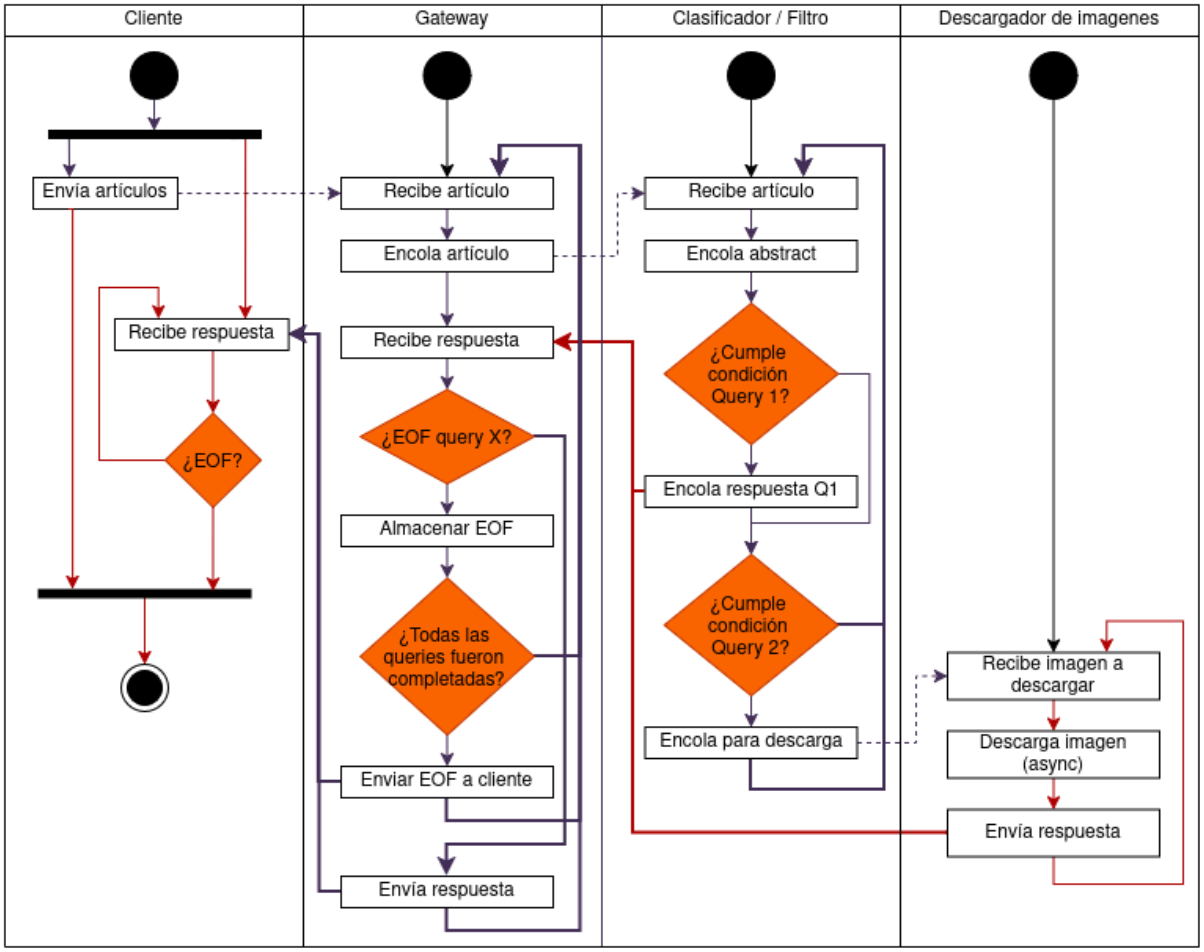
El cliente tiene dos flujos de procesamiento, uno para el envío de artículos y otro para la recepción de respuestas. Por otro lado, el gateway tiene un flujo similar para el ingreso de artículos en el sistema y otro para recibir las respuestas a las queries, enviarlas al cliente y finalizar la comunicación cuando todas las solicitudes fueron respondidas.

Luego el clasificador recibe el artículo a través de su cola, ejecutando los siguientes pasos

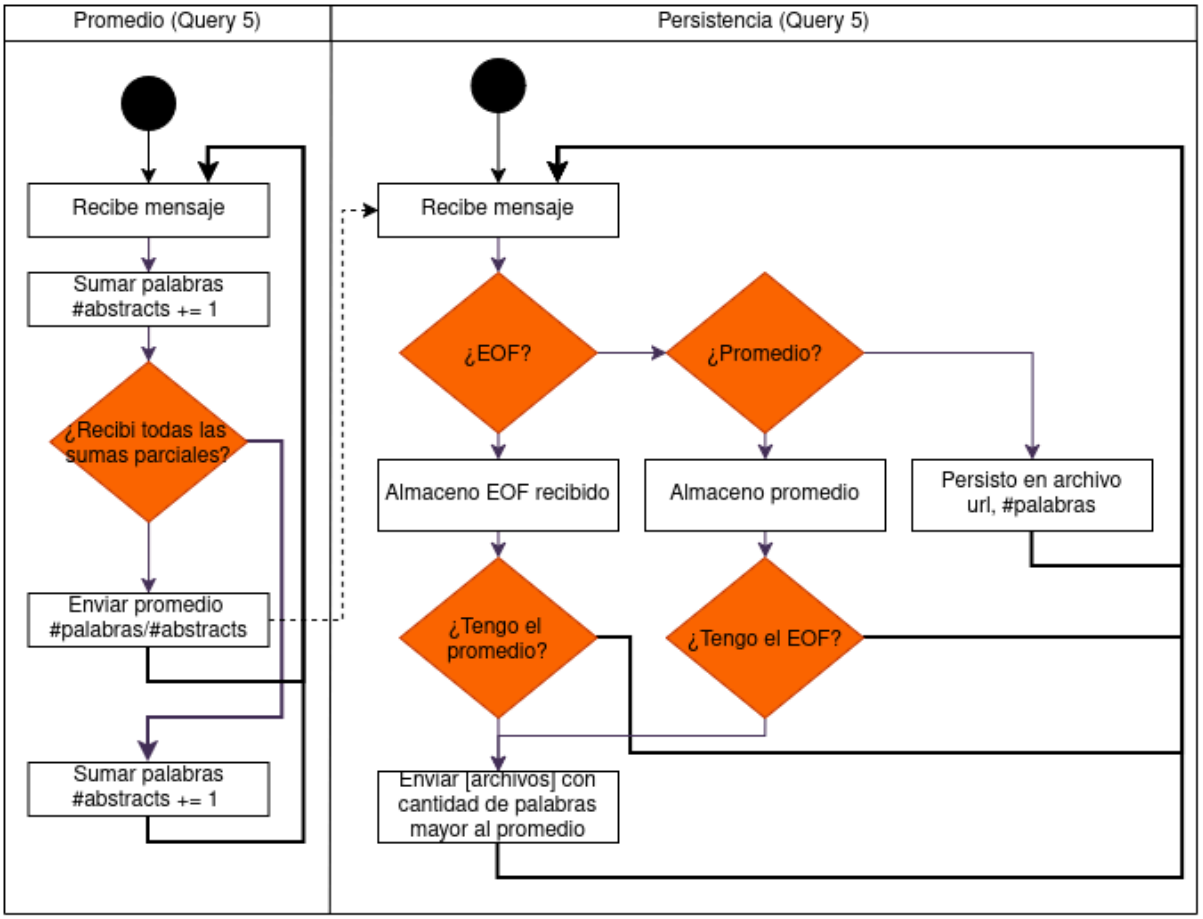
1. Encolar abstract para ser preprocesado por alguno de los transformadores
2. Si el artículo cumple las condiciones de la primera query (escrito en inglés hasta el 2020 inclusive) se encola la información solicitada para resolver dicha query.
3. Si el artículo cumple las condiciones de la segunda query (incluye sección biography o biographie, hasta el 2020) se encola para la descarga de la imágen.

Vale mencionar que este filtro y encolado de información específica reduce la dimensionalidad de la información en el sistema, reduciendo de tempranamente la carga sobre los subsistemas siguientes.

Finalmente el transformador recibe el abstract y aplica el criterio de conteo de palabras, resultando en la lista de palabras para la tercera, cuarta y quinta query. Para la tercera y cuarta query envía la lista de palabras y para la quinta query envía el largo de la lista de palabras, ahorrando recursos en la cola y subsistema dedicado para dicha query.

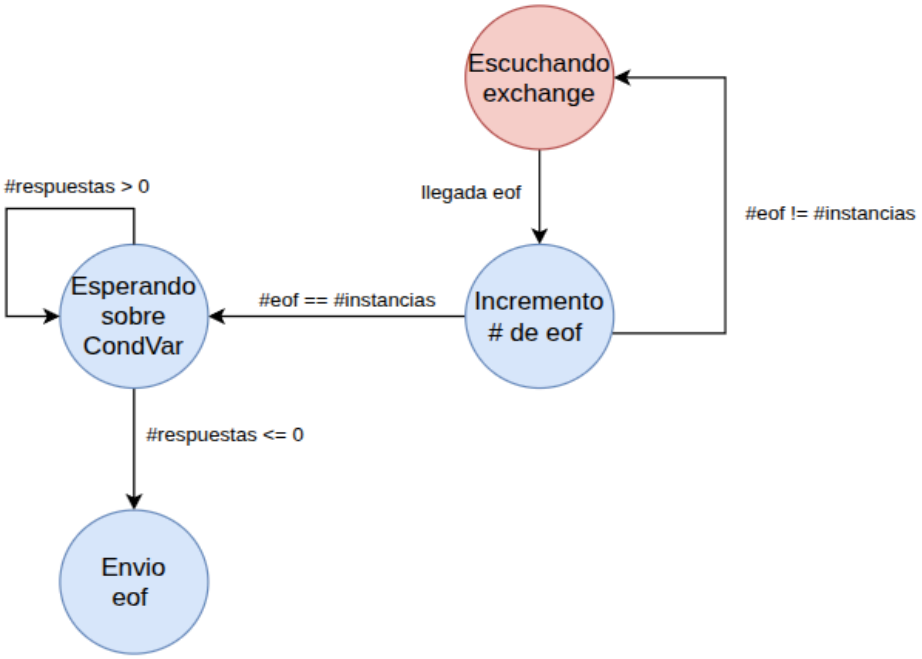


La cuarta query, el top de palabras más utilizadas, se resolverá con un sistema similar al del trabajo práctico de coordinación, por medio de nodos SUM, AGGREGATE y JOIN.
Para la tercera query los agregadores del top de palabras almacenarán, además del conteo por palabra, la cantidad de idiomas en la que la palabra fue utilizada, devolviendo la primera coincidencia de la palabra en ambos idiomas.
Por último, para resolver la query 5 se va a encolar por fecha el id de usuario, url y la cantidad de palabras del abstract. Si la fecha es mayor al 31/12/2020 se almacena url y cantidad de palabras en un archivo append-only por usuario. Caso contrario, es encolado para el cálculo del promedio.

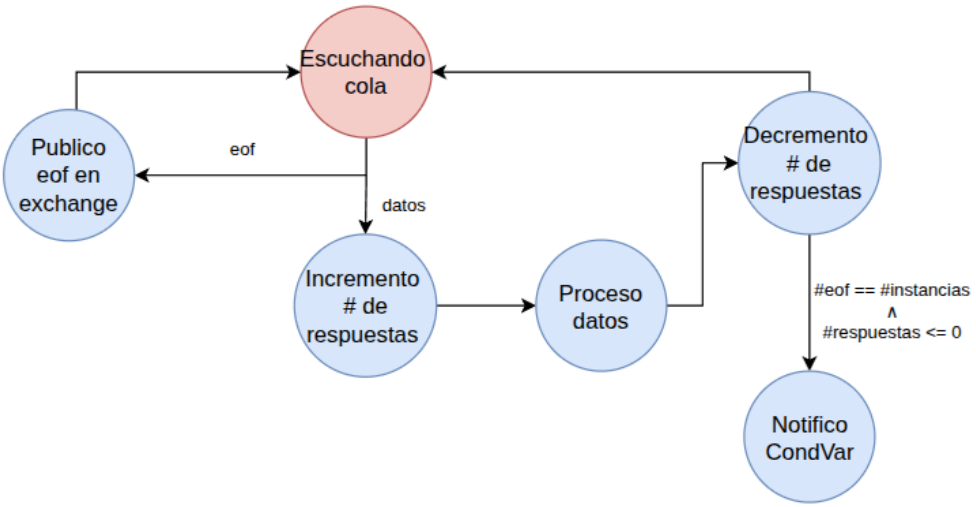


Sincronización de etapas

Cada instancia posee 2 hilos, uno de control y uno de datos.
El hilo de control se mantiene bloqueado escuchando en un exchange donde se publicaran los EOF. Al llegar un EOF actúa de la siguiente manera:



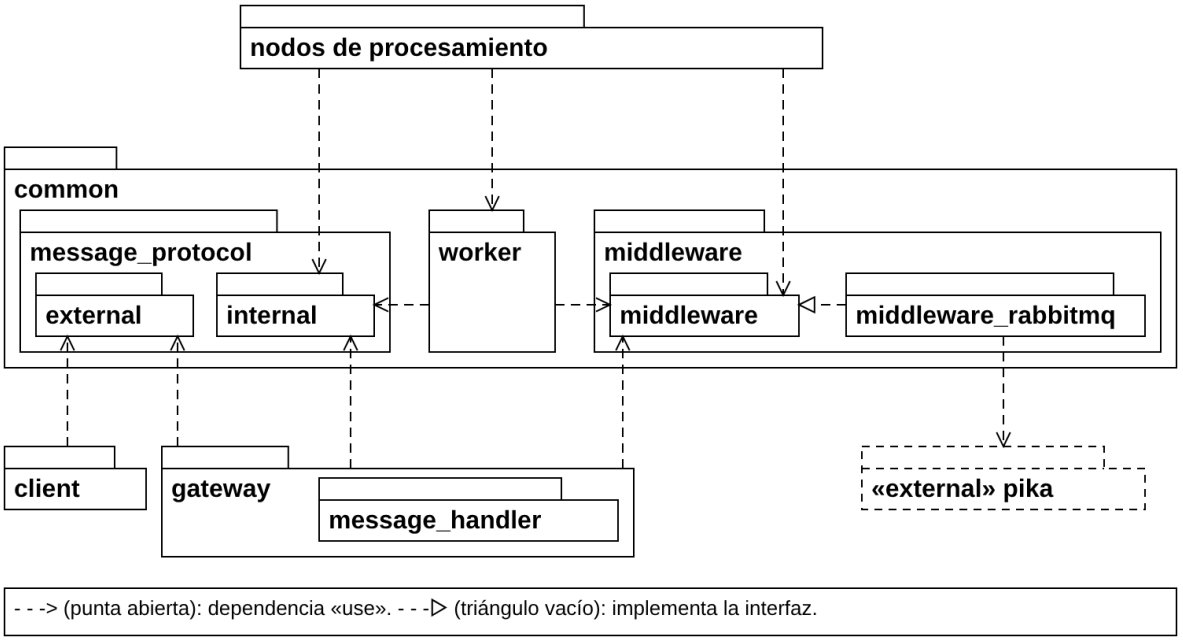
La etapa anterior va a enviar **#instancias** eof a la siguiente etapa, la cual va a estar esperando por esa cantidad de eofs para poder propagar el eof a la siguiente etapa. Además se espera que la cantidad de respuestas sea cero para confirmar que el hilo de datos haya terminado de procesar todos los mensajes recibidos.
El hilo de control se mantiene bloqueado escuchando sobre la cola de datos de entrada, donde recibe mensajes de datos y los eofs. Actúa de la siguiente manera:



Vista de Desarrollo

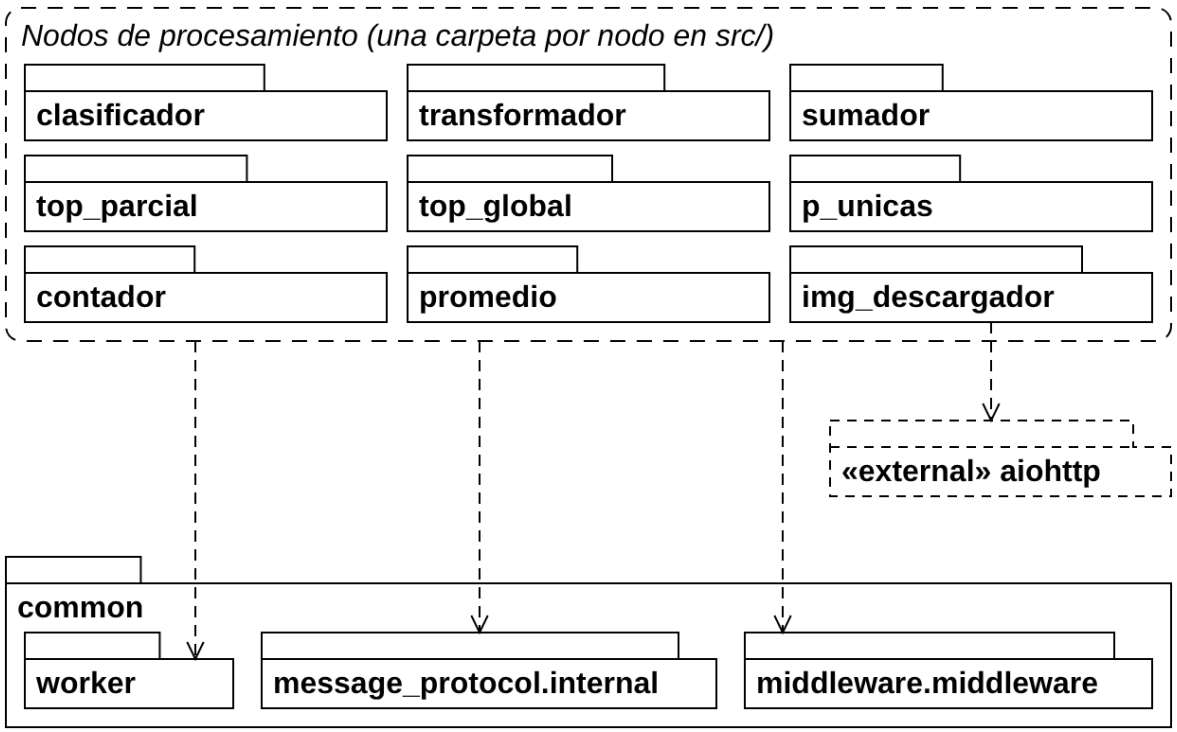
El código se organiza en tres grupos de paquetes. `client` contiene el programa del cliente, que lee el dataset, envía los artículos y guarda los resultados. `gateway` contiene el servidor TCP que atiende a los clientes, ingresa los artículos al sistema y les devuelve los

resultados. Los nodos de procesamiento (Figura 3) contienen un paquete por cada tipo de nodo. Todos ellos dependen de `common`, que agrupa el código compartido (Figura 2). La única dependencia externa del sistema con el broker es la librería `pika`, y la usa solamente el middleware.



- message_protocol define los mensajes del sistema y está dividido en dos partes. external es el protocolo binario por TCP entre `client` y `gateway`, junto con su serializador. **internal** define los mensajes entre nodos (tipos y campos de mensaje, y el EOF por cliente). El cliente usa solo la parte externa, los nodos solo la interna y el gateway las dos.
- middleware implementa la comunicación por grupos. El módulo middleware declara la interfaz (`MessageMiddleware` con sus variantes de cola y exchange) y middleware_rabbitmq la implementa sobre `pika`. Los nodos y el gateway dependen solo de la interfaz, por lo que el resto del sistema no depende de RabbitMQ.
- worker es la base común de todos los nodos de procesamiento: el ciclo de consumo, el estado y el conteo de EOF por userId, y el graceful quit ante SIGTERM.

Dentro de `gateway`, message_handler traduce entre el protocolo externo y el interno, y asigna los identificadores de cliente.



- - -> (punta abierta): dependencia «use».
Las flechas que salen del marco valen para todos los nodos.
Los nodos no dependen entre sí: se comunican solo a través del middleware.

Cada nodo es una carpeta en `src/` con su propio `main.py`, que lee de variables de entorno la cantidad de réplicas y las colas que usa. Los nombres coinciden con los de la Vista de Despliegue.

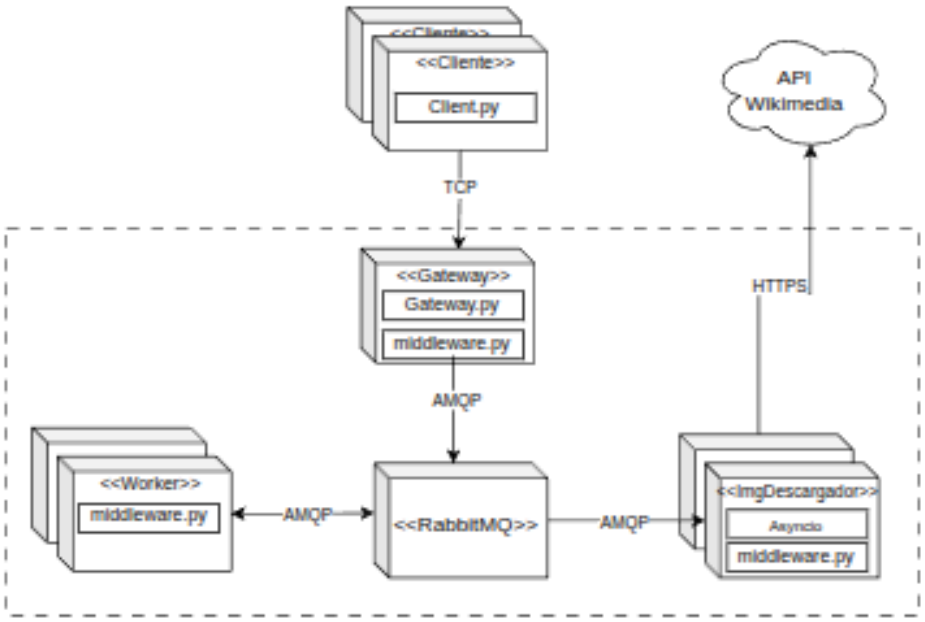
- clasificador**:** resuelve la query 1, filtra los artículos de la query 2 y envía el abstract al transformador.
- transformador**:** normaliza el abstract, lo separa en palabras, quita las stopwords y deduplica.
- img_descargador**:** descarga las imágenes de Wikimedia, con reintentos.
- sumador**:** suma parcialmente las palabras por usuario e idioma (query 4).
- top_parcial**:** acumula el conteo por palabra e idioma y, con el EOF, emite el top 20 parcial y las palabras únicas.
- top_global**:** junta los tops parciales de cada usuario y calcula el top 20 global (query 4) y encuentra las coincidencias de la tercera quert,
- contador**:** cuenta las palabras de cada artículo para la query 5. Para los artículos hasta 2020 acumula suma y cantidad parciales; para los posteriores guarda url y cantidad por usuario y, con el promedio, devuelve los que lo superan.
- promedio**:** junta las sumas parciales de cada usuario y calcula el promedio (query 5).

El sistema contará con los siguientes componentes:

1. Gateway-Controlador: Mantendrá las conexiones con los clientes, manejando la recepción de solicitudes - envío de respuestas e insertando las solicitudes válidas en el sistema para su procesamiento.
2. Clasificador/Filtro inicial: Se encargará de responder la primera query y filtrar los artículos que cumplan las condiciones de la segunda query para la descarga de la imagen
3. Descargador de imágenes para query 2
4. Transformador de abstract a palabras: Recibirá cada abstract, devolviendo la lista de palabras según el criterio del enunciado para su uso posterior en la tercera, cuarta y quinta query.
5. Query 3: Solo se requiere un comparador de palabras (detallado en las próximas secciones)
6. Query 4: Dado que es un escenario muy similar al trabajo de coordinación, se realiza una implementación similar.
7. Query 5: Se tendrá un par sumador-join para obtener el promedio de palabras en artículos a partir del 2021 y nodos de persistencia que almacenan la lista de url, cantidad de palabras para artículos hasta el 2020 inclusive.
   .

Vista de Despliegue

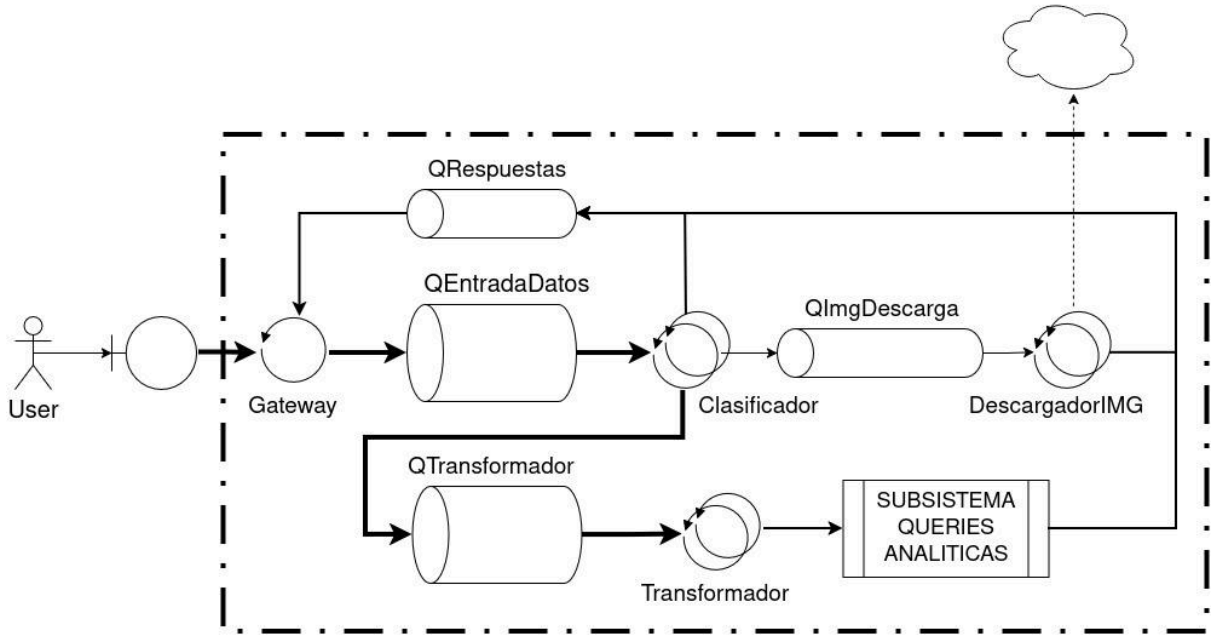
En el siguiente diagrama se puede observar la distribución física de los componentes del sistema. Cada componente se ejecutará dentro de su contenedor docker y el conjunto se orquestara mediante Docker Compose. Los contenedores deberían poder repartirse entre una o varias máquinas, lo que permitiría aprovechar entornos multi computadora.



El cliente queda por fuera del sistema y se comunica únicamente con el gateway por medio de TCP. El gateway servirá como frontera recibiendo los artículos de cada cliente, los presenta al sistema y devuelve los resultados de todas las queries.
Dentro del sistema ningún nodo se comunica de manera “directa” con otro nodo, todos los mensajes pasan por medio del broker (rabbitmq). Este desacople permite agregar o quitar réplicas de cada nodo sin necesidad de modificar las demás.
Los nodos con profundidad doble son replicables y se puede configurar la cantidad dependiendo de la cantidad de clientes y el volumen de consultas y datos a procesar.
El descargador de imágenes es el único nodo que se comunica con el servicio de wikimedia para obtener las imágenes durante la query 2 mediante https.
El nodo Worker puede ser de tipo agregadorTop, clasificador, cuentaProm, joinTop, promedio, sumTop o transformador.

Diagrama de Robustez

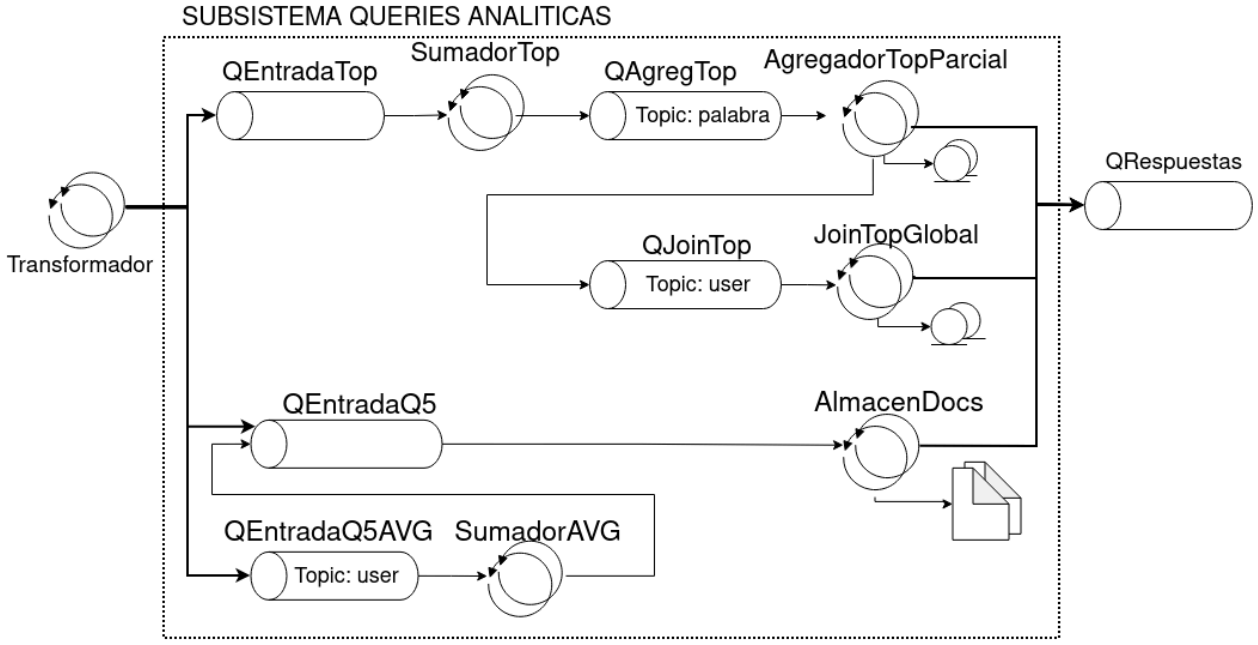
El siguiente diagrama corresponde a la arquitectura general del sistema. Puede observarse como el gran volumen de carga es dirigido tempranamente a los transformadores para una reducción temprana de dicha carga[1]. La primera y segunda query son resueltas entre el clasificador y el descargador, mientras que las restantes son resueltas por el subsistema de queries analítica.



1. El top de palabras es resuelto por medio de la tríada sum-aggregate-join, teniendo afinidad por palabras con los aggregate y afinidad por usuario en los join, permitiendo tener varios nodos join que habiliten la escalabilidad horizontal del sistema. Se realizará la sumatoria para inglés y francés por separado, almacenando en dos diccionarios palabra-conteo.

[1] Considerar que el total de palabras del dataset es 11857372, mientras que el total de palabras sin repetir y filtrando stopwords es 360298. Se adjunta notebook editado en los anexos.

2. Las palabras compartidas entre artículos en inglés-francés serán detectadas por los aggregate, quienes almacenan el conteo por palabra y la cantidad de idiomas en las que fue utilizada.
3. Para la quinta query se tendrán dos colas, una para calcular el promedio de cantidad de palabras de artículos hasta 2020 inclusive y otra para almacenar la cantidad de palabras de artículos a partir del 2021. El promedio se calculará por medio de nodos SUM que realizará la suma y conteo total. Mientras que los artićulos a partir de 2021 ,se enviarán a los nodos de persistencia con afinidad por usuario, los cuales almacenarán los mismos en archivos por cliente en modo append que, luego de recibir el promedio y el EOF marcando el fin de envío de artículos, leerán el archivo devolviendo los artículos que cumplan la condición.



Anexos:

Conteo y total de palabras

El siguiente fragmento de código puede utilizarse para contabilizar la cantidad de palabras únicas en el dataset y el total de palabras entre todos los abstract, tal de poder observar el factor de reducción entre la información entrante y saliente de los transformadores.
Debe ser reemplazado por la sección equivalente en el notebook original.

```
# Word and count of the 20 most common words in the abstract
TOP_SIZE = 20
WORD_REGEX = r'\b\w+\b'
NUMBER_REGEX = r'[+-]?((\d+\.?\d*)|(\.\d+))'
# Source and discussion:
https://stackoverflow.com/questions/12643009/regular-expression-for-floating-poi
nt-numbers/42629198#42629198

def extract_non_stopwords_from_abstract(abstract):
    if not isinstance(abstract, str):
        return []
    words = re.findall(WORD_REGEX, abstract.lower())
    return [word for word in words if word not in stop_words and not
re.fullmatch(NUMBER_REGEX, word)]

def extract_word_count(abstract):
    if not isinstance(abstract, str):
        return []
    words = re.findall(WORD_REGEX, abstract.lower())
    return len(words)

stop_words = set(stopwords.words('english')) | set(stopwords.words('french'))
words_counter = collections.Counter()
total_words = 0
for abstract in wiki_entry_df['abstract'].dropna():
    total_words += extract_word_count(abstract)
    non_stop_words = extract_non_stopwords_from_abstract(abstract)
    words_counter.update(set(non_stop_words))
print(f"Total words: {total_words}\nUnique words: {len(words_counter)}")
pd.DataFrame(words_counter.most_common(TOP_SIZE), columns=["word", "count"])
```